

DevOps Interview

Question & Answer Guide

Sanket Ajay Chopade · DevOps Engineer

Hisan Labs | AWS · Kubernetes · Terraform · CI/CD · Docker

16 Sections	200+ Questions	Resume Tailored
-----------------------	--------------------------	---------------------------

This guide provides structured, resume-specific answers to every question category an interviewer is likely to ask for a 0–1 year DevOps Engineer profile. Each answer is crafted from real internship experience and project work.

Table of Contents

Section 1	Resume Introduction Questions
Section 2	Internship-Based Questions — Hisan Labs
Section 3	AWS Interview Questions
Section 4	Terraform Questions
Section 5	Docker Questions
Section 6	Kubernetes Questions
Section 7	GitHub Actions & GitOps Questions
Section 8	Monitoring & Observability Questions
Section 9	Linux Questions
Section 10	Bash & Python Questions
Section 11	Security Questions
Section 12	Project-Specific Questions
Section 13	AWS Generative AI Questions
Section 14	Behavioral Questions
Section 15	Situational Questions
Section 16	HR Questions

Section 1 | Resume Introduction Questions

Q1. Tell me about yourself.

- My name is Sanket Chopade, a B.Tech graduate from Rashtrasant Tukadoji Maharaj Nagpur University.
 - I recently completed a DevOps Engineer internship at Hisan Labs in Pune (Sep 2025 – May 2026).
 - During the internship I worked on a microservices-based e-commerce application deployed on AWS.
 - I have hands-on experience with AWS (EKS, IAM, VPC, RDS, S3), Terraform, Kubernetes, Docker, Jenkins, GitHub Actions, and Argo CD.
 - I also worked on monitoring using Prometheus, Grafana, Datadog, and AWS CloudWatch.
 - I am a quick learner who is passionate about automation, cloud infrastructure, and building reliable delivery pipelines.
-

Q2. Walk me through your resume.

- I started my B.Tech in 2021 and graduated in 2025 from Nagpur University.
 - During my final year I joined Hisan Labs as a DevOps Engineer Intern.
 - There, I was part of a 4-member DevOps team supporting an e-commerce platform on AWS.
 - My core responsibilities included building Jenkins pipelines, writing Terraform modules, managing EKS clusters, and setting up monitoring.
 - I also worked on two independent projects: a Go cloud-native GitOps pipeline and a serverless game hosting platform on EKS Fargate.
 - My skills span the full DevOps lifecycle — infrastructure provisioning, containerization, CI/CD, and observability.
-

Q3. Why did you choose DevOps as a career?

- I was drawn to DevOps because it sits at the intersection of development and operations — bridging the gap between writing code and running it reliably.
 - During college I explored Linux, scripting, and cloud services and realized I enjoyed building automated systems more than just writing application code.
 - DevOps also has a visible impact: faster releases, fewer failures, and better reliability directly benefit end users.
 - The ecosystem of tools — Terraform, Kubernetes, CI/CD — excited me, and the steep learning curve keeps the work challenging and rewarding.
-

Q4. What did you learn during your internship at Hisan Labs?

- End-to-end CI/CD pipeline design using Jenkins with GitHub webhooks and SonarQube integration.
 - Terraform-based IaC — writing reusable modules, managing remote state, and handling multi-environment deployments.
 - Kubernetes cluster management on Amazon EKS — Deployments, Services, Ingress, HPA, ConfigMaps, and Secrets.
 - Real-world monitoring: configuring Prometheus exporters, building Grafana dashboards, and interpreting CloudWatch and Datadog metrics.
 - AWS IAM best practices — creating roles, policies, and implementing least privilege.
 - Team collaboration skills: code reviews, documentation, and coordinating deployments across a 4-member team.
-

Q5. What were your day-to-day responsibilities?

- Maintaining and improving Jenkins pipelines for infrastructure, frontend, and backend deployments.
 - Writing and reviewing Terraform code to provision or update AWS resources.
 - Monitoring application health via Grafana and Datadog dashboards, and responding to alerts.
 - Reviewing pull requests, running pipeline checks, and supporting deployments to different environments.
 - Troubleshooting Kubernetes pod failures, service misconfigurations, and Ingress issues on EKS.
 - Automating operational tasks using Bash and Python scripts.
 - Updating operational runbooks and infrastructure documentation.
-

Q6. What DevOps tools are you most comfortable with?

- CI/CD: Jenkins and GitHub Actions — I designed pipelines from scratch and troubleshoot pipeline failures.
 - IaC: Terraform — I wrote reusable modules for AWS resource provisioning.
 - Containers & Orchestration: Docker and Kubernetes on Amazon EKS.
 - Cloud: AWS — daily use of EKS, IAM, VPC, S3, RDS, ALB, and CloudWatch.
 - Monitoring: Prometheus and Grafana for metrics, Datadog for APM.
 - GitOps: Argo CD for continuous delivery, syncing K8s state from Git.
-

Q7. Which project are you most proud of and why?

- I am most proud of the Go Cloud-Native GitOps Delivery Pipeline project.
 - I built an end-to-end pipeline where a developer pushes code to GitHub, GitHub Actions runs tests and builds a Distroless Docker image, and Argo CD automatically syncs the deployment to EKS.
 - It demonstrated the full GitOps principle — Git as the single source of truth for Kubernetes state.
 - Using Distroless images reduced the attack surface and image size significantly.
 - The project sharpened my understanding of the complete software delivery lifecycle from commit to production.
-

Q8. What challenges did you face during your internship?

- Managing Terraform state across team members — resolved by configuring an S3 backend with DynamoDB state locking.
 - Debugging intermittent pod CrashLoopBackOff errors on EKS — resolved by analyzing container logs and fixing environment variable misconfigurations via ConfigMaps.
 - Pipeline failures due to SonarQube quality gate violations — worked with developers to fix code smells and reduce technical debt.
 - Understanding IAM permission boundaries when setting up IRSA for EKS workloads — resolved after careful study of OIDC trust relationships.
-

Q9. Why should we hire you for a DevOps role?

- I have real internship experience — not just theoretical knowledge — working on production-grade AWS infrastructure.
 - I have hands-on exposure to the full DevOps toolchain: Terraform, Kubernetes, Jenkins, Argo CD, Prometheus, Grafana, and Docker.
 - I am a self-starter who independently designed and delivered two end-to-end projects.
 - I am comfortable with ambiguity, can troubleshoot under pressure, and document my work for team knowledge sharing.
 - I bring fresh energy, up-to-date knowledge of modern DevOps practices, and a strong willingness to grow.
-

Q10. Where do you see yourself in the next 3–5 years?

- In 3 years I want to become a proficient DevOps or Site Reliability Engineer, owning cloud infrastructure for large-scale systems.
 - I aim to deepen expertise in platform engineering — building internal developer platforms and service meshes.
 - I also want to pursue AWS certifications (Solutions Architect, DevOps Engineer Professional) to formalize my knowledge.
 - In 5 years I aspire to lead a DevOps or infrastructure team, driving adoption of best practices and mentoring junior engineers.
-

Section 2 | Internship-Based Questions — Hisan Labs

Q1. Explain the architecture of the e-commerce application you worked on.

- It was a microservices-based e-commerce application running on AWS.
 - The frontend and backend services were containerized with Docker and deployed on Amazon EKS.
 - Data persistence was handled by Amazon RDS (relational database) and S3 (object storage).
 - Traffic was routed through an Application Load Balancer (ALB) and an Ingress controller inside EKS.
 - Infrastructure was provisioned using Terraform (VPCs, subnets, security groups, EKS cluster, RDS).
 - A separate CI/CD pipeline existed for Terraform infra, frontend, and backend — all managed by Jenkins.
-

Q2. Explain the Jenkins pipelines you created.

- Pipeline 1 — Terraform Infra: Triggered on infra code changes. Stages: checkout → terraform init → plan → apply (with approval gate).
 - Pipeline 2 — Frontend: checkout → npm install → SonarQube scan → Docker build & push → deploy to EKS.
 - Pipeline 3 — Backend: checkout → Maven build → SonarQube scan → Docker build & push → kubectl rollout to EKS.
 - All pipelines used GitHub webhooks for automatic triggering on push/PR events.
 - Credentials (AWS keys, Docker Hub token, SonarQube token) were stored in Jenkins Credentials Manager.
-

Q3. How did you handle deployment failures?

- For Kubernetes deployments, a rolling update strategy ensured no downtime — if health checks failed, the rollout was automatically paused.
 - I used kubectl rollout undo to revert to the previous ReplicaSet immediately.
 - Jenkins pipeline failures triggered Slack/email notifications, allowing quick identification.
 - For Terraform failures, I analyzed the plan output, fixed the root cause, and re-ran apply.
 - Post-mortem notes were documented to prevent repeat failures.
-

Q4. How did GitHub webhooks trigger Jenkins?

- A webhook was configured in the GitHub repository settings pointing to the Jenkins server URL (/github-webhook/).
 - On every push or pull request event, GitHub sent a POST request to Jenkins.
 - Jenkins had the GitHub plugin installed, which parsed the payload and triggered the matching pipeline.
 - Branch filtering was applied so only pushes to main or specific branches triggered production deployments.
-

Q5. How did you manage Terraform state?

- We used an S3 bucket as the remote backend to store the terraform.tfstate file.
 - DynamoDB table was configured for state locking to prevent concurrent modifications by team members.
 - Each environment (dev, staging, prod) had its own state file path in S3 (e.g., env/dev/terraform.tfstate).
 - The backend configuration was defined in the backend.tf file and initialised with terraform init.
-

Q6. What Kubernetes resources did you use?

- Deployment — to manage pod lifecycle and rolling updates.
 - Service — ClusterIP for internal communication, NodePort/LoadBalancer for external traffic.
 - Ingress — to route external HTTP/HTTPS traffic to backend services with path-based rules.
 - ConfigMap — to inject non-sensitive configuration (env vars, config files) into pods.
 - Secret — to inject sensitive data (DB passwords, API keys) as base64-encoded values.
 - HorizontalPodAutoscaler (HPA) — to auto-scale pods based on CPU/memory metrics.
-

Q7. How did HPA scale workloads?

- HPA watches CPU and memory metrics via the Kubernetes Metrics Server.
 - A target utilization threshold (e.g., 70% CPU) was defined in the HPA spec.
 - When average utilization exceeded the threshold, HPA increased the replica count (up to maxReplicas).
 - When load dropped, HPA scaled down after a cooldown period (stabilizationWindowSeconds).
 - This ensured cost efficiency and availability during traffic spikes on the e-commerce platform.
-

Q8. What metrics did you monitor?

- Infrastructure: CPU utilization, memory usage, disk I/O, network throughput — via CloudWatch and Prometheus node exporter.
 - Kubernetes: pod restarts, container CPU/memory, failed deployments — via kube-state-metrics.
 - Application: request latency, error rates (4xx/5xx), throughput — via application-level Prometheus metrics.
 - Database: RDS connections, query latency, slow queries — via CloudWatch RDS metrics.
 - ALB: request count, target response time, unhealthy host count — via CloudWatch.
-

Section 3 | AWS Interview Questions

Q1. Difference between IAM User, Role, and Policy.

- IAM User: A permanent identity (person or application) with long-term credentials (username/password or access keys).
 - IAM Role: A temporary identity assumed by AWS services, EC2 instances, Lambda functions, or cross-account entities. No permanent credentials.
 - IAM Policy: A JSON document that defines permissions (Allow/Deny). Policies are attached to users, groups, or roles.
 - Best practice: prefer roles over users for AWS service-to-service access to avoid storing long-term credentials.
-

Q2. What is least privilege access?

- Granting only the minimum permissions an entity needs to perform its specific tasks — nothing more.
 - Example: an EKS worker node needs EC2, ECR, and ELB permissions — not S3 or RDS.
 - Implemented using fine-grained IAM policies with specific Actions, Resources, and Conditions.
 - Reduces blast radius in case of credential compromise.
 - We applied this during internship when creating IAM roles for Jenkins, EKS nodes, and IRSA.
-

Q3. What is a VPC?

- A Virtual Private Cloud is a logically isolated section of the AWS cloud where you define your own network.
 - You control IP address ranges (CIDR blocks), subnets, route tables, internet gateways, and security.
 - We created a custom VPC with public subnets (for ALB/NAT) and private subnets (for EKS nodes and RDS) using Terraform.
 - Public subnet: has a route to the Internet Gateway. Private subnet: routes outbound traffic through a NAT Gateway.
-

Q4. Explain Security Groups vs NACLs.

- Security Groups: Virtual firewalls at the instance level. Stateful — if inbound is allowed, response traffic is automatically allowed. Rules apply to ENIs.
 - NACLs (Network Access Control Lists): Operate at the subnet level. Stateless — inbound and outbound rules are evaluated independently.
 - Security Groups support only Allow rules; NACLs support both Allow and Deny.
 - We used security groups to restrict access to RDS (only from EKS node SG) and to EKS API (only from office CIDR).
-

Q5. What is Amazon RDS Multi-AZ?

- Multi-AZ automatically provisions a standby DB instance in a different Availability Zone.
 - AWS synchronously replicates data to the standby.
 - In case of primary failure, RDS automatically fails over to the standby — typically within 1–2 minutes.
 - The endpoint DNS name remains the same, so applications reconnect transparently.
 - We used Multi-AZ on the production RDS instance for the e-commerce platform to ensure high availability.
-

Q6. What is Amazon EKS?

- Amazon Elastic Kubernetes Service is a managed Kubernetes control plane service.
 - AWS manages the control plane (API server, etcd, scheduler) — you manage worker nodes (EC2 or Fargate).
 - Integrates with IAM, VPC, ALB, CloudWatch, and ECR natively.
 - We used EKS for all containerized microservices deployments during the internship.
 - Benefit over self-managed K8s: no need to manage control plane HA, patching, or etcd backups.
-

Q7. What is an ALB vs NLB?

- ALB (Application Load Balancer): Layer 7. Routes based on URL path, host headers, HTTP methods. Ideal for HTTP/HTTPS microservices.
 - NLB (Network Load Balancer): Layer 4. Routes based on TCP/UDP port. Ultra-low latency, handles millions of requests/sec.
 - We used ALB with the AWS Load Balancer Controller on EKS to route HTTP traffic to services via Ingress rules.
 - NLB is preferred for gaming, IoT, or latency-sensitive workloads.
-

Q8. What is S3 and its storage classes?

- Amazon S3 is an object storage service offering virtually unlimited scalability.
 - S3 Standard: Frequently accessed data. High durability (11 9s) and availability.
 - S3 Standard-IA: Infrequently accessed data. Lower cost but retrieval fee.
 - S3 Glacier: Archival storage. Retrieval time from minutes to hours.
 - S3 Intelligent-Tiering: Automatically moves objects between tiers based on access patterns.
 - We used S3 for Terraform remote state storage and as a static asset store for the e-commerce frontend.
-

Q9. How does IAM Roles for Service Accounts (IRSA) work?

- IRSA enables Kubernetes service accounts to assume IAM roles using OIDC federation.
 - An OIDC identity provider is associated with the EKS cluster.
 - An IAM role is created with a trust policy that allows the specific Kubernetes service account (identified by cluster OIDC URL, namespace, and SA name) to assume it.
 - Pods annotated with that service account automatically receive temporary AWS credentials.
 - Used in the Serverless Game Hosting project to allow the AWS Load Balancer Controller to manage ALB resources securely.
-

Section 4 | Terraform Questions

Q1. What is Infrastructure as Code? Why Terraform?

- IaC is the practice of managing and provisioning infrastructure through code rather than manual processes.
 - Benefits: version control, repeatability, peer review, auditability, and consistency across environments.
 - We chose Terraform because it is cloud-agnostic, has a large module ecosystem, and uses declarative HCL syntax.
 - Terraform vs CloudFormation: Terraform works across multiple cloud providers; CloudFormation is AWS-only.
-

Q2. Explain the Terraform workflow.

- terraform init: Downloads providers and modules, sets up the backend.
 - terraform plan: Shows a preview of changes (what will be created, modified, or destroyed).
 - terraform apply: Applies the changes to real infrastructure after confirmation.
 - terraform destroy: Tears down all managed resources.
 - In our Jenkins pipeline: init → plan (output saved) → apply (on approval).
-

Q3. What is terraform.tfstate and why is it important?

- The state file is a JSON file that maps Terraform configuration to real-world resources.
 - Terraform uses it to determine what exists, what needs to change, and what to destroy.
 - If state is lost: Terraform has no record of existing resources and may try to recreate them, causing duplicates or conflicts.
 - We stored state in S3 (remote backend) so the entire team could share it safely.
-

Q4. What is state locking?

- State locking prevents two Terraform operations from running concurrently against the same state file.
 - Without locking, two team members running terraform apply simultaneously could corrupt the state.
 - We used a DynamoDB table as the lock backend — Terraform writes a lock entry before modifying state and releases it after.
-

Q5. Difference between count and for_each.

- count: Creates N identical resources indexed by integer. If you remove a middle item, all subsequent resources are re-created.
 - for_each: Creates resources keyed by a map or set. Removing one key only destroys that specific resource — safer for production.
 - We prefer for_each for environments and IAM roles to avoid accidental destruction.
 - Example: for_each = { dev = 'us-east-1', prod = 'us-west-2' } creates two differently-named resources.
-

Q6. What are Terraform modules?

- A module is a reusable, self-contained collection of Terraform resources with defined inputs (variables) and outputs.
 - During internship we created modules for: VPC, EKS cluster, RDS instance, and security groups.
 - Modules promoted consistency across environments — dev/staging/prod all used the same VPC module with different variable values.
 - Root module calls child modules; child modules can also be sourced from the Terraform Registry.
-

Q7. What are Terraform workspaces?

- Workspaces allow you to manage multiple state files within a single Terraform configuration.
 - Each workspace (default, dev, staging, prod) has its own state file.
 - Useful for lightweight environment separation — not recommended for large-scale multi-environment setups where separate directories/repos are preferred.
 - Switch with: `terraform workspace new dev` / `terraform workspace select dev`.
-

Q8. What is terraform import?

- `terraform import` allows you to bring existing manually-created infrastructure under Terraform management.
 - Command: `terraform import aws_s3_bucket.example my-bucket-name`
 - After import, Terraform knows about the resource in state, but you still need to write the matching HCL configuration.
 - Useful when taking over legacy infrastructure or onboarding existing resources into IaC.
-

Section 5 | Docker Questions

Q1. What is Docker and why use containers?

- Docker is a platform for building, packaging, and running applications in containers.
 - Containers bundle the application code, runtime, libraries, and config into a single portable image.
 - Containers share the host OS kernel — lighter and faster to start than VMs.
 - Eliminates 'works on my machine' problems by ensuring consistent runtime environments.
 - We containerized all microservices at Hisan Labs for consistent deployment across dev, staging, and prod.
-

Q2. Difference between CMD and ENTRYPOINT.

- ENTRYPOINT: Defines the fixed executable that always runs. It cannot be overridden with docker run arguments.
 - CMD: Provides default arguments to the ENTRYPOINT or a default command. Can be overridden at runtime.
 - Best practice: use ENTRYPOINT for the main binary and CMD for default flags.
 - Example: ENTRYPOINT ["/app/server"] and CMD ["--port", "8080"] — port can be overridden at runtime.
-

Q3. What are multi-stage builds and why use Distroless images?

- Multi-stage builds use multiple FROM instructions in a Dockerfile — you build in one stage and copy only the binary/artifact to a minimal final image.
 - This keeps the final image small and free from build tools (compilers, package managers), reducing attack surface.
 - Distroless images (from Google) contain only the application runtime — no shell, no OS utilities.
 - Combined, they dramatically reduce image size and the number of CVEs (vulnerabilities).
 - Used in the Go GitOps project: Go binary was compiled in a Go builder image, then copied into a Distroless Go image.
-

Q4. How do you debug containers?

- `kubectl exec -it -- /bin/sh` — opens an interactive shell inside a running container.
 - `kubectl logs` — view stdout/stderr from the container.
 - `docker run -it --entrypoint /bin/sh` — inspect image contents before deployment.
 - For Distroless images (no shell): use `kubectl debug` to attach an ephemeral debug container.
 - Check Kubernetes events with `kubectl describe pod` for scheduling and image pull issues.
-

Q5. What is Docker networking?

- Bridge (default): Containers on the same Docker host can communicate via a virtual bridge network.
 - Host: Container shares the host's network stack — no isolation. Used for performance-sensitive workloads.
 - None: No network. Container is fully isolated.
 - Overlay: For multi-host communication in Docker Swarm.
 - In Kubernetes (EKS), container networking is managed by the VPC CNI plugin — each pod gets a VPC IP address.
-

Section 6 | Kubernetes Questions

Q1. Explain Kubernetes cluster architecture.

- Control Plane: API Server (entry point for all K8s operations), etcd (distributed key-value store for cluster state), Scheduler (assigns pods to nodes), Controller Manager (runs control loops).
 - Worker Nodes: kubelet (communicates with API server, manages pods on the node), kube-proxy (manages networking rules), container runtime (Docker/containerd).
 - On EKS: AWS manages the control plane. We managed the worker node groups (EC2 managed node groups).
-

Q2. What is a Pod vs Deployment vs ReplicaSet?

- Pod: Smallest deployable unit in Kubernetes. Contains one or more containers sharing network and storage.
 - ReplicaSet: Ensures a specified number of pod replicas are running at all times. Handles pod failures.
 - Deployment: Higher-level abstraction that manages ReplicaSets. Provides rolling updates, rollbacks, and desired state management.
 - We always create Deployments — never bare Pods — for production workloads.
-

Q3. What is a Service in Kubernetes?

- A Service provides a stable network endpoint (DNS name + IP) to access a set of pods.
 - ClusterIP: Internal-only access. Used for service-to-service communication inside the cluster.
 - NodePort: Exposes the service on each node's IP at a static port. Used for testing.
 - LoadBalancer: Provisions a cloud load balancer (ALB/NLB on AWS). We used this for the Ingress controller.
 - ExternalName: Maps a service to an external DNS name.
-

Q4. What is an Ingress?

- Ingress is a Kubernetes resource that manages external HTTP/HTTPS traffic routing to services.
 - It uses an Ingress Controller (we used the AWS Load Balancer Controller) to provision an ALB.
 - Supports path-based routing (/api → backend service, / → frontend service) and host-based routing.
 - TLS termination can be configured at the Ingress level using ACM certificates on AWS.
-

Q5. Pod is not starting — what will you check?

- `kubectl describe pod` : Look at Events section for scheduling, image pull, or resource errors.
 - `kubectl logs --previous`: Check logs from the previous crashed container.
 - Common causes: ImagePullBackOff (wrong image tag/registry auth), CrashLoopBackOff (application crash on start), Pending (insufficient node resources), OOMKilled (exceeded memory limit).
 - Check ConfigMaps/Secrets referenced by the pod for missing or incorrect values.
 - Verify the node has enough CPU/memory using `kubectl describe node`.
-

Q6. Difference between HPA and Cluster Autoscaler.

- HPA (Horizontal Pod Autoscaler): Scales the number of pod replicas based on CPU/memory or custom metrics.
 - Cluster Autoscaler: Scales the number of worker nodes in the cluster. When pods cannot be scheduled due to insufficient node resources, CA adds a new node.
 - They work together: HPA scales pods → if nodes are full, CA adds nodes → pods get scheduled.
 - We configured HPA on EKS; on EKS Fargate (Game Hosting project), node scaling is fully managed by AWS.
-

Q7. What kubectl commands do you use regularly?

- `kubectl get pods/svc/deployment -n` — list resources.
 - `kubectl describe pod/node` — detailed info and events.
 - `kubectl logs -f` — stream logs.
 - `kubectl exec -it -- /bin/sh` — shell into a pod.
 - `kubectl apply -f .yaml` — apply configuration.
 - `kubectl rollout status/undo deployment/` — check or roll back a deployment.
 - `kubectl top pods/nodes` — resource usage.
-

Section 7 | GitHub Actions & GitOps Questions

Q1. What is GitHub Actions?

- GitHub Actions is a CI/CD platform natively integrated with GitHub repositories.
 - Workflows are defined as YAML files in `.github/workflows/`.
 - Triggered by events: `push`, `pull_request`, `schedule`, `workflow_dispatch`.
 - Jobs run on GitHub-hosted runners (`ubuntu-latest`, `windows-latest`) or self-hosted runners.
 - We used GitHub Actions in the Go GitOps project to build Docker images, run tests, and push to ECR.
-

Q2. What is GitOps and how does Argo CD work?

- GitOps is a deployment methodology where Git is the single source of truth for infrastructure and application state.
 - Desired state is declared in Git (Kubernetes manifests/Helm charts). Automated tooling reconciles the cluster to match Git.
 - Argo CD is a declarative GitOps tool for Kubernetes. It continuously monitors Git repos and syncs changes to the cluster.
 - Argo CD detects drift between the live cluster state and Git, and either auto-syncs or alerts the team.
 - In the Go GitOps project: GitHub Actions built and pushed a new image tag → updated the Helm values file in Git → Argo CD detected the change and rolled out the new version.
-

Q3. Difference between Jenkins and Argo CD.

- Jenkins is a general-purpose CI tool — it runs scripts, builds code, runs tests, and can deploy.
 - Argo CD is a specialized CD tool focused solely on Kubernetes GitOps delivery.
 - Jenkins uses push-based deployment (pipeline pushes to K8s); Argo CD uses pull-based (it pulls from Git and reconciles).
 - Argo CD provides drift detection, rollback from Git history, and a UI showing cluster sync status.
 - We used Jenkins for CI (build/test/image) and Argo CD for CD (Kubernetes deployment reconciliation).
-

Q4. How do you manage secrets in GitHub Actions?

- Secrets are stored in GitHub Settings → Secrets and Variables → Actions.
 - Referenced in workflows as `${{ secrets.SECRET_NAME }}`.
 - Common secrets: AWS credentials, Docker Hub tokens, SonarQube tokens.
 - For production workloads, prefer OIDC-based authentication (GitHub → AWS) over long-lived access keys.
-

Section 8 | Monitoring & Observability Questions

Q1. What is observability? Difference from monitoring.

- Monitoring: Tracking predefined metrics and alerting on known failure conditions.
 - Observability: The ability to understand the internal state of a system from its external outputs (metrics, logs, traces).
 - Observability answers 'why is it failing?'; monitoring answers 'is it failing?'.
 - Three pillars of observability: Metrics (what), Logs (what happened in detail), Traces (where in the request chain).
 - We implemented metrics (Prometheus/Grafana/Datadog) and logs (CloudWatch) at Hisan Labs.
-

Q2. Explain Prometheus architecture.

- Prometheus is a pull-based time-series metrics system.
 - Prometheus Server: Scrapes metrics from configured targets at defined intervals (`scrape_interval`).
 - Exporters: Applications that expose metrics in Prometheus format — `node_exporter` (system), `kube-state-metrics` (K8s objects), application-level exporters.
 - Alertmanager: Receives alerts from Prometheus and routes them to receivers (Slack, PagerDuty, email).
 - PromQL: Query language used to write alert rules and Grafana dashboard queries.
-

Q3. What is Grafana used for?

- Grafana is a visualization and dashboarding tool that queries multiple data sources.
 - We connected Grafana to Prometheus for infrastructure/K8s metrics and CloudWatch for AWS-native metrics.
 - Dashboards I built: node CPU/memory overview, EKS pod health, Jenkins pipeline duration, and ALB request rate.
 - Grafana supports alert rules that can send notifications to Slack/PagerDuty when thresholds are breached.
-

Q4. How would you investigate high CPU usage?

- Step 1 — Identify: `kubectl top pods -n` to find the pod with the highest CPU. `kubectl top nodes` for node-level view.
 - Step 2 — Inspect: `kubectl describe pod` shows resource requests/limits; check if limits are too low.
 - Step 3 — Logs: `kubectl logs` for application-level errors or tight loops.
 - Step 4 — Profiling: Use application profiling tools (pprof for Go) to pinpoint the hot code path.
 - Step 5 — Remediate: Increase resource limits, scale replicas with HPA, optimize the code, or add caching.
-

Q5. What alerts would you configure?

- Pod CrashLoopBackOff — alert if restart count > 3 in 5 minutes.
 - Node CPU > 85% — sustained for 5+ minutes.
 - Deployment unavailable replicas > 0 — immediate alert.
 - ALB 5xx error rate > 1% — sustained for 2+ minutes.
 - RDS connection count approaching `max_connections` limit.
 - Disk usage on nodes > 80%.
 - Jenkins build failure rate — alert if > 2 consecutive failures.
-

Section 9 | Linux Questions

Q1. Explain the Linux boot process.

- 1. BIOS/UEFI: Performs POST and hands off to the bootloader.
 - 2. GRUB (Bootloader): Loads the Linux kernel into memory.
 - 3. Kernel Initialization: Kernel decompresses, initializes hardware, and mounts the root filesystem.
 - 4. init/systemd: PID 1 process. Reads unit files and starts system services.
 - 5. Login shell: System reaches the configured target (multi-user.target or graphical.target).
-

Q2. How do you check running processes?

- `ps aux` — snapshot of all running processes with CPU and memory usage.
 - `top / htop` — interactive, real-time process viewer.
 - `pgrep` — returns PID of processes matching the name.
 - `kill -9` — forcefully terminate a process.
 - `systemctl status` — check systemd service status.
-

Q3. What is a cron job?

- A scheduled task managed by the cron daemon on Linux.
 - Defined in a crontab file using the format: minute hour day month weekday command.
 - Example: `0 2 * * * /home/sanket/backup.sh` runs the backup script at 2 AM daily.
 - `crontab -e` to edit, `crontab -l` to list existing jobs.
 - We used cron jobs to schedule Bash scripts for log rotation and health check reporting.
-

Q4. How do you troubleshoot a slow server?

- `top/htop` — identify CPU-hungry processes.
 - `free -m` — check if memory is exhausted (look for swap usage).
 - `df -h` — check disk space. `iostat` — check disk I/O wait.
 - `netstat -tuln` or `ss -s` — check network connections.
 - `dmesg | tail` — check kernel messages for hardware errors.
 - Check application logs and systemd service logs with `journalctl -xe`.
-

Q5. Explain file permissions and `chmod 755`.

- Linux file permissions: three sets (owner, group, others) each with read (4), write (2), execute (1).
 - `chmod 755` = owner: `rwx` (7), group: `r-x` (5), others: `r-x` (5).
 - $7 = 4+2+1$ = read+write+execute. $5 = 4+1$ = read+execute.
 - Used for executable scripts that others can read and run but not modify.
 - `chmod -R 755 /app` to apply recursively. `chown user:group` to change ownership.
-

Section 10 | Bash & Python Questions

Q1. What automation did you build using Bash?

- Automated Docker image cleanup scripts to remove dangling images from Jenkins agents.
 - Health-check scripts that polled API endpoints and reported status to Slack.
 - Wrapper scripts for common kubectl commands to simplify team workflows.
 - Log rotation and archiving scripts scheduled via cron on EC2 instances.
 - Environment setup scripts for bootstrapping new developer machines with required tools.
-

Q2. How do you pass arguments to a Bash script?

- \$1, \$2, ... — positional parameters. \$0 is the script name.
 - \$@ — all arguments as separate strings. \$# — count of arguments.
 - Example: ./deploy.sh prod us-east-1 → \$1=prod, \$2=us-east-1.
 - Use getopts for named flags: ./deploy.sh -e prod -r us-east-1.
-

Q3. How would you parse a JSON file in Python?

- import json; with open('file.json') as f: data = json.load(f)
 - Access fields: data['key'] or data.get('key', default).
 - For API responses: response = requests.get(url); data = response.json().
 - For complex structures: use jq command-line tool in Bash scripts or json module in Python.
-

Q4. How would you automate AWS tasks using Python?

- Use the boto3 SDK: import boto3; client = boto3.client('ec2', region_name='us-east-1')
 - Example — list running EC2 instances:
ec2.describe_instances(Filters=[{'Name':'instance-state-name','Values':['running']}])
 - Automate S3 backups: s3.upload_file(file_path, bucket_name, object_key)
 - IAM automation: create/delete users, attach policies, rotate access keys.
 - We used Python scripts to automate tag compliance checks across EC2 and S3 resources.
-

Section 11 | Security Questions

Q1. What is DevSecOps?

- DevSecOps integrates security practices into every stage of the DevOps lifecycle — not as an afterthought.
 - Security shifts left: vulnerabilities are caught at code commit (SonarQube), image build (Trivy), and before deployment.
 - Tools we used: SonarQube for code quality/security in CI, Trivy for container image vulnerability scanning.
 - IAM least privilege, secrets management (Jenkins credentials), and IRSA for K8s workloads are security-by-design choices.
-

Q2. What issues can SonarQube detect?

- Code smells: maintainability issues (duplicated code, complex functions).
 - Bugs: logic errors likely to cause runtime failures.
 - Vulnerabilities: security issues such as SQL injection risks, hardcoded credentials, insecure deserialization.
 - Security hotspots: code requiring manual review (e.g., use of eval(), weak encryption).
 - Coverage gaps: lines not covered by unit tests.
 - We enforced a quality gate in Jenkins — builds failed if the vulnerability count exceeded the threshold.
-

Q3. What is Trivy and how do you use it?

- Trivy is an open-source vulnerability scanner for container images, filesystems, and IaC files.
 - It scans OS packages (apt, yum) and application dependencies (pip, npm, Maven) for known CVEs.
 - Command: trivy image — outputs a severity-categorized vulnerability report.
 - Integrated into CI pipeline: if CRITICAL vulnerabilities are found, the build fails and the image is not pushed.
 - Also supports scanning Terraform files and Kubernetes manifests for misconfigurations.
-

Q4. How do you secure Kubernetes Secrets?

- By default, Secrets are base64-encoded (not encrypted) in etcd — they must be protected.
 - Enable encryption at rest for etcd using AWS KMS (EKS supports this via envelope encryption).
 - Use AWS Secrets Manager or HashiCorp Vault with External Secrets Operator to fetch secrets at runtime.
 - Apply RBAC to restrict which service accounts can read specific Secrets.
 - Avoid committing base64-encoded Secrets to Git — use sealed-secrets or External Secrets Operator.
-

Q5. How would you secure a CI/CD pipeline?

- Store all credentials in a secrets manager (Jenkins Credentials, GitHub Actions Secrets) — never hardcoded.
 - Use OIDC federation (GitHub → AWS) to avoid long-lived access keys.
 - Integrate SAST (SonarQube) and image scanning (Trivy) as mandatory pipeline stages.
 - Enforce branch protection rules — require PRs, passing CI checks, and code reviews before merging.
 - Apply least privilege IAM roles for the pipeline service account.
 - Audit pipeline run history and log all deployments for compliance.
-

Section 12 | Project-Specific Questions

Q1. Explain the Go Cloud-Native GitOps Delivery Pipeline.

- A Go web application is hosted on GitHub. Any push to main triggers a GitHub Actions workflow.
 - Workflow stages: code checkout → unit tests → multi-stage Docker build (Distroless) → Trivy image scan → push to ECR with a commit-SHA tag.
 - After push, a second workflow step updates the image tag in the Helm values.yaml file in a separate config repo.
 - Argo CD watches the config repo. On detecting the new image tag, it performs a rolling update on EKS.
 - Kubernetes resources: Deployment, Service, Ingress (ALB), Helm-managed.
 - Rollback: revert the image tag in Git → Argo CD reverts the cluster state automatically.
-

Q2. Why use Distroless images?

- Distroless images contain only the application runtime — no shell, no package manager, no OS utilities.
 - Dramatically reduces the image attack surface: fewer packages = fewer CVEs.
 - Smaller image size improves pull speed and reduces storage costs.
 - Forces good practices: no debugging inside containers in production; use ephemeral debug containers instead.
-

Q3. Explain the Serverless Game Hosting Platform.

- Deployed a containerized 2048 game on Amazon EKS using AWS Fargate compute — no EC2 nodes to manage.
 - Fargate is serverless Kubernetes: AWS provisions and manages the underlying compute for each pod.
 - Installed the AWS Load Balancer Controller via Helm to provision an ALB for ingress traffic to the game.
 - Configured IAM OIDC integration between the EKS cluster and AWS to enable IRSA.
 - Created an IAM role with the required ALB permissions, bound it to the LBC's service account via IRSA.
 - Used eksctl and kubectl for cluster management and application deployment.
-

Q4. What is IRSA and why use OIDC?

- IRSA (IAM Roles for Service Accounts) grants Kubernetes pods temporary AWS credentials via IAM role assumption.
 - OIDC (OpenID Connect) is the federation mechanism — EKS acts as an OIDC identity provider.
 - AWS STS validates the Kubernetes JWT token against the OIDC provider and issues temporary credentials.
 - This eliminates the need to store AWS access keys inside pods or use EC2 instance profile credentials shared across all workloads.
 - Each microservice gets its own minimal IAM role — enforcing least privilege at the pod level.
-

Section 13 | AWS Generative AI Questions

Q1. What is Amazon Bedrock?

- Amazon Bedrock is a fully managed service that provides access to foundation models (FMs) from leading AI companies via a unified API.
 - Supported models include Anthropic Claude, Amazon Titan, Meta Llama, Mistral, and more.
 - No need to manage infrastructure — just call the API with a prompt and model ID.
 - Supports RAG (Retrieval Augmented Generation) via Bedrock Knowledge Bases.
 - My exposure was introductory — exploring the console, invoking Claude models, and understanding the API structure.
-

Q2. What is Amazon Q Developer?

- Amazon Q Developer is an AI-powered coding assistant integrated into IDEs (VS Code, JetBrains) and the CLI.
 - It generates code suggestions, explains code, writes unit tests, and helps debug.
 - Also assists DevOps engineers with CloudFormation/CDK template generation and AWS CLI command suggestions.
 - Useful for understanding unfamiliar AWS services and quickly scaffolding IaC code.
-

Q3. Difference between Amazon Bedrock and SageMaker.

- Bedrock: Managed access to pre-trained foundation models. No ML training infrastructure to manage. Best for inference/generation tasks.
 - SageMaker: Full ML platform for training, tuning, deploying, and monitoring custom ML models.
 - Bedrock = consume AI. SageMaker = build and train AI.
 - For DevOps use cases (AI-powered log analysis, anomaly detection), Bedrock APIs are simpler to integrate.
-

Section 14 | Behavioral Questions

Q1. Tell me about a difficult problem you solved.

- Situation: Our Terraform state became inconsistent when two engineers applied changes simultaneously.
 - Task: Restore consistent state without destroying production resources.
 - Action: I analyzed the divergence using terraform plan, manually reconciled the state file, and implemented DynamoDB-based state locking immediately.
 - Result: No resources were lost. Going forward, locking prevented similar incidents.
-

Q2. Describe a situation where a deployment failed.

- Situation: A backend service rollout on EKS resulted in pods stuck in CrashLoopBackOff.
 - Task: Restore service quickly with minimal downtime.
 - Action: Ran kubectl logs and kubectl describe — found a missing environment variable (DB connection string) in the Deployment spec. Immediately ran kubectl rollout undo to restore the previous working version.
 - Result: Service restored in under 3 minutes. I then added a config validation step to the Jenkins pipeline to catch missing env vars before deploy.
-

Q3. Describe a time you learned a new technology quickly.

- Situation: I was assigned to set up Argo CD for GitOps delivery with zero prior hands-on experience.
 - Task: Have Argo CD deploying to EKS within one sprint.
 - Action: I studied the official docs, followed a hands-on tutorial, and had a working setup in 3 days. I documented the process for the team.
 - Result: Argo CD became the standard CD tool for the team. The GitOps project was delivered on schedule.
-

Q4. How do you prioritize tasks?

- I use urgency vs impact as the primary filter — production incidents (high urgency, high impact) always come first.
 - I communicate with the team daily to align priorities, especially when unplanned tasks appear.
 - I maintain a task list and time-box research tasks so they do not block delivery.
 - If tasks conflict in priority, I escalate to my team lead rather than guessing.
-

Section 15 | Situational Questions

Q1. Production deployment failed. What would you do?

- 1. Stay calm. Immediately notify stakeholders of the issue and ETA for resolution.
 - 2. Check Kubernetes rollout status: `kubectl rollout status deployment/`.
 - 3. If pods are failing: `kubectl logs` and `kubectl describe` to identify the root cause.
 - 4. Rollback immediately: `kubectl rollout undo deployment/`.
 - 5. Validate rollback — confirm all pods are running and the service is healthy.
 - 6. Root cause analysis: determine what changed (code, config, infra) and fix in a new release.
 - 7. Document the incident — timeline, impact, cause, and corrective action.
-

Q2. Terraform state file gets corrupted. How would you recover?

- 1. Stop all terraform operations immediately to prevent further corruption.
 - 2. Restore the last known good state from S3 versioning (we always had versioning enabled on the state bucket).
 - 3. Run `terraform plan` to compare restored state vs actual infrastructure.
 - 4. Use `terraform import` to re-import any resources missing from state.
 - 5. Investigate the root cause — usually concurrent applies or manual state edits.
 - 6. Ensure DynamoDB locking is enabled to prevent recurrence.
-

Q3. A Kubernetes pod is in `CrashLoopBackOff`. What will you check?

- `kubectl logs --previous` — inspect the crash logs of the previous container run.
 - `kubectl describe pod` — check Events section for `OOMKilled`, failed mounts, or image pull errors.
 - Verify environment variables via `ConfigMaps` and `Secrets` are correct.
 - Check resource limits — if memory limit is too low, the container may be `OOMKilled`.
 - Reproduce locally: `docker run` the same image with the same env vars to test outside K8s.
-

Q4. CPU utilization suddenly spikes. What is your approach?

- Step 1 — Scope: Is it node-level or pod-level? (`kubectl top nodes` / `kubectl top pods`)
 - Step 2 — Identify: Find the offending pod. Check recent deployments — was a new release just rolled out?
 - Step 3 — Application: Review application logs for tight loops, runaway goroutines, or unexpected traffic.
 - Step 4 — Scale: If load is legitimate, HPA should trigger — verify HPA status with `kubectl get hpa`.
 - Step 5 — Remediate: Roll back the recent deployment if it's the cause, or throttle incoming traffic at the ALB.
-

Q5. Argo CD is not syncing changes. What would you check?

- Check Argo CD UI: Is the app in an Out of Sync or Error state?
 - Verify the Git repo URL and branch/path configuration in the Argo CD Application spec.
 - Check if Argo CD has connectivity to the Git repo (credentials, private repo access).
 - Inspect the sync operation logs in the Argo CD UI or `argocd` app logs.
 - Ensure the target cluster's kubeconfig is still valid (token not expired).
 - Check if auto-sync is enabled: `argocd app set --sync-policy automated`.
-

Q6. A security vulnerability is found in a Docker image. What would you do?

- 1. Identify severity: Is it CRITICAL/HIGH? Assess if it is exploitable in the running context.
 - 2. Stop promotion: Prevent the vulnerable image from being deployed to production.
 - 3. Patch: Update the base image to the patched version (e.g., distroless:latest or alpine:3.x).
 - 4. Rebuild and rescan: Rebuild the image and run Trivy again to confirm the CVE is resolved.
 - 5. Deploy the patched image through the standard CI/CD pipeline.
 - 6. If already in production: assess risk, apply a runtime mitigation if available, and fast-track the patch.
 - 7. Add the CVE to the pipeline's deny list to block future deployments of the vulnerable package version.
-

Section 16 | HR Questions

Q1. Why DevOps?

- I am passionate about building systems that allow teams to deliver software faster, reliably, and at scale.
 - DevOps is a force-multiplier — one well-designed pipeline or infrastructure module can save thousands of hours of manual work.
 - The breadth of the field — cloud, containers, security, networking, scripting — keeps me engaged and continuously learning.
-

Q2. Why should we hire you?

- I have real, hands-on experience — not just certifications. I deployed production infrastructure, managed live EKS clusters, and built pipelines used by real developers.
 - I am proactive: I take ownership, document my work, and seek to improve processes beyond my assigned tasks.
 - I am adaptable and fast-learning — I picked up Argo CD and IRSA independently within days when needed.
 - I bring a security-first mindset with hands-on experience in SonarQube, Trivy, and IAM least privilege.
-

Q3. What are your strengths and weaknesses?

- Strengths: Strong troubleshooting ability, systematic approach to problem-solving, hands-on experience with modern DevOps toolchain, good documentation habits.
 - Weakness: I sometimes spend extra time trying to fully understand a concept before moving forward. I am working on balancing depth of learning with delivery speed by setting timebox limits for research tasks.
-

Q4. Do you have any questions for us?

- What does the on-call rotation look like for the DevOps team?
 - What cloud maturity stage is the infrastructure at, and what are the top priorities for the next 6 months?
 - How do you handle knowledge sharing within the team — do you have internal tech talks or documentation standards?
 - What does the career growth path look like for a DevOps Engineer here?
-

Good luck, Sanket!

Remember: every answer should be tied back to your real experience at Hisan Labs or your projects. Use the STAR method (Situation, Task, Action, Result) for behavioral questions. Confidence, honesty about your learning journey, and enthusiasm for DevOps will set you apart.